

Linux 驱动 Makefile 详解

正点原子 i.MX6ULL · 01~05 驱动例程

chrdevbase / led / newchrled / dtsled / gpioled

第一章 Kbuild Makefile 通用结构

这 5 个驱动例程的 Makefile 结构完全一致，属于 Linux 内核的 **Kbuild**（内核构建系统）标准模板。理解了这个模板，后续所有驱动的 Makefile 都可以直接套用。

1.1 完整模板

```
KERNELDIR := /home/zuozhongkai/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
CURRENT_PATH := $(shell pwd)

obj-m := <模块名>.o

build: kernel_modules

kernel_modules:
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules

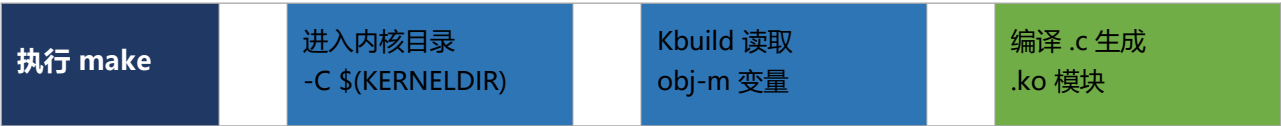
clean:
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

1.2 逐行详解

关键字 / 语法	含义与说明
KERNELDIR :=	指定 Linux 内核源码树的绝对路径。Kbuild 需要进入内核目录读取编译规则、头文件和配置。实际使用时需改为自己开发环境中内核的路径。
:= 赋值	Make 的立即展开赋值（Simply Expanded），右侧在读取时就被求值，区别于 = 的延迟展开。此处没有变量引用，效果相同；但习惯上路径用 := 更安全。

关键字 / 语法	含义与说明
CURRENT_PATH := \$(shell pwd)	\$(shell ...) 是 Make 的 Shell 函数，执行括号内的 Shell 命令并将结果作为字符串返回。pwd 返回当前工作目录的绝对路径，赋值给 CURRENT_PATH，告诉 Kbuild 驱动源码在哪里。
obj-m := <模块名>.o	Kbuild 专用变量。 obj-m 表示将此目标编译为可动态加载的内核模块（.ko 文件）。 若写 obj-y 则编译进内核镜像（不生成 .ko）。 右侧写 .o 文件名，Kbuild 会自动寻找同名 .c 源文件进行编译。
build: kernel_modules	build 是自定义的默认目标（执行 make 时默认调用）。 它依赖 kernel_modules 目标，相当于 make build → 触发 make kernel_modules。
\$(MAKE) -C \$(KERNELDIR) M=\$(CURRENT_PATH) modules	\$(MAKE)：递归调用 make，保持与顶层相同的 make 版本和参数。 -C \$(KERNELDIR)：切换到内核源码目录执行。 M=\$(CURRENT_PATH)：告诉 Kbuild 外部模块（驱动源码）的路径。 modules：内核 Makefile 中的目标，触发外部模块编译，最终生成 .ko 文件。
\$(MAKE) -C \$(KERNELDIR) M=\$(CURRENT_PATH) clean	与上面相同，但目标改为 clean，让内核 Kbuild 清理本模块编译产生的中间文件（.o、.ko、.mod.c、Module.symvers 等）。

1.3 编译流程图



为什么不直接 gcc 编译？
Linux 驱动运行在内核态，必须使用内核的编译器配置、头文件和符号表。Kbuild 保证驱动与内核版本完全匹配，直接 gcc 编译会缺少 kernel.h 等关键头文件。

第二章 01_chrdevbase — 字符设备基础驱动

chrdevbase 是 Linux 驱动学习的第一个例程，演示最基础的字符设备注册流程：手动分配主设备号、实现 open / read / write / release 四个文件操作接口。

1.1 Makefile 内容

```
KERNELDIR := /home/zuozhongkai/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
CURRENT_PATH := $(shell pwd)

obj-m := chrdevbase.o

build: kernel_modules
```

kernel_modules:

```
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules
```

clean:

```
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

1.2 与通用模板的区别

变量	本例程的值	说明
obj-m	chrdevbase.o	编译 chrdevbase.c, 生成 chrdevbase.ko 内核模块
其余变量	与通用模板完全相同	KERNELDIR / CURRENT_PATH / 规则均不变

1.3 使用说明

命令	作用
make	编译生成 chrdevbase.ko
make clean	清理所有编译产物
insmod chrdevbase.ko	在开发板上加载模块
rmmod chrdevbase	卸载模块
dmesg tail	查看内核 printk 输出

学习重点:

此例程 Makefile 与后续例程完全相同, 区别只在 .c 源文件。chrdevbase.c 中通过 register_chrdev() 静态注册主设备号, 是理解字符设备驱动框架的起点。

第三章 02_led — LED 字符设备驱动

led 例程在 chrdevbase 基础上加入了真实硬件操作: 通过 ioremap 将 i.MX6ULL 的 GPIO 物理地址映射到内核虚拟地址, 然后操作寄存器控制 LED 亮灭。

2.1 Makefile 内容

```
KERNELDIR := /home/zuozhongkai/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
```

```
CURRENT_PATH := $(shell pwd)
```

```
obj-m := led.o
```

```
build: kernel_modules
```

```
kernel_modules:

$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules

clean:

$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

2.2 与通用模板的区别

变量	本例程的值	说明
obj-m	led.o	编译 led.c, 生成 led.ko 内核模块
其余变量	与通用模板完全相同	KERNELDIR / CURRENT_PATH / 规则均不变

2.3 使用说明

命令	作用
make	编译生成 led.ko
make clean	清理所有编译产物
insmod led.ko	在开发板上加载模块
rmmod led	卸载模块
dmesg tail	查看内核 printk 输出

学习重点:

led.c 展示了 ioremap / iounmap 的用法, 这是驱动访问硬件寄存器的标准方式。Makefile 本身无变化, 但此例程标志着从"纯软件框架"进入"真实硬件操作"阶段。

第四章 03_newchrled — 新字符设备框架

newchrled 使用 alloc_chrdev_region (动态分配设备号) + cdev 结构体 + class_create / device_create 自动在 /dev 下创建设备节点, 是现代 Linux 驱动的标准写法。

3.1 Makefile 内容

```
KERNELDIR := /home/zuozhongkai/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
CURRENT_PATH := $(shell pwd)

obj-m := newchrled.o

build: kernel_modules
```

kernel_modules:

```
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules
```

clean:

```
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

3.2 与通用模板的区别

变量	本例程的值	说明
obj-m	newchrled.o	编译 newchrled.c, 生成 newchrled.ko 内核模块
其余变量	与通用模板完全相同	KERNELDIR / CURRENT_PATH / 规则均不变

3.3 使用说明

命令	作用
make	编译生成 newchrled.ko
make clean	清理所有编译产物
insmod newchrled.ko	在开发板上加载模块
rmmod newchrled	卸载模块
dmesg tail	查看内核 printk 输出

新旧对比:

旧方式 (chrdevbase) : register_chrdev() 静态注册, 需手动 mknod 创建节点。

新方式 (newchrled) : alloc_chrdev_region + class/device 自动注册, /dev 节点自动出现。

工程实际驱动几乎全部使用新方式。

第五章 04_dtsled — 设备树 LED 驱动

dtsled 引入设备树 (Device Tree) 概念: GPIO 信息不再硬编码在 .c 文件中, 而是在 .dts 设备树文件中描述, 驱动通过 of_xxx API 从设备树节点读取属性。

4.1 Makefile 内容

```
KERNELDIR := /home/zuozhongkai/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
```

```
CURRENT_PATH := $(shell pwd)
```

```
obj-m := dtsled.o
```

```
build: kernel_modules

kernel_modules:
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules

clean:
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

4.2 与通用模板的区别

变量	本例程的值	说明
obj-m	dtsled.o	编译 dtsled.c, 生成 dtsled.ko 内核模块
其余变量	与通用模板完全相同	KERNELDIR / CURRENT_PATH / 规则均不变

4.3 使用说明

命令	作用
make	编译生成 dtsled.ko
make clean	清理所有编译产物
insmod dtsled.ko	在开发板上加载模块
rmmod dtsled	卸载模块
dmesg tail	查看内核 printk 输出

设备树的意义：

同一份驱动代码可适配不同硬件（只改 .dts 不改 .c），是 Linux 3.x 之后嵌入式驱动的核心设计思想。Makefile 照旧不变——编译 .ko 的方式与硬件描述方式无关。

第六章 05_gpioled — GPIO 子系统驱动

gpioled 在 dtsled 基础上用 GPIO 子系统接管硬件操作：不再手动 ioremap 操作寄存器，而是调用 gpio_request / gpio_direction_output / gpio_set_value，由内核 GPIO 子系统统一管理引脚。

5.1 Makefile 内容

```
KERNELDIR := /home/zuozhongkai/linux/IMX6ULL/linux/temp/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
CURRENT_PATH := $(shell pwd)

obj-m := gpioled.o
```

```
build: kernel_modules
```

```
kernel_modules:
```

```
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) modules
```

```
clean:
```

```
$(MAKE) -C $(KERNELDIR) M=$(CURRENT_PATH) clean
```

5.2 与通用模板的区别

变量	本例程的值	说明
obj-m	gpioled.o	编译 gpioled.c, 生成 gpioled.ko 内核模块
其余变量	与通用模板完全相同	KERNELDIR / CURRENT_PATH / 规则均不变

5.3 使用说明

命令	作用
make	编译生成 gpioled.ko
make clean	清理所有编译产物
insmod gpioled.ko	在开发板上加载模块
rmmod gpioled	卸载模块
dmesg tail	查看内核 printk 输出

为什么推荐 GPIO 子系统？

手动 ioremap 直接操作寄存器：移植性差，不同芯片地址不同。
GPIO 子系统：统一接口，换芯片只改设备树，驱动代码基本不动。
这是工程中 GPIO 驱动的最终形态。

第七章 移植到自己的环境：修改 KERNELDIR

例程中的 KERNELDIR

是课程作者开发机上的路径，在自己的环境中必须修改为本地内核源码的实际路径，否则 make 会直接报错找不到目录。

7.1 查找内核路径

如果使用正点原子配套内核（解压后）：

```
KERNELDIR := /home/<你的用户名>/linux/linux-imx-rel_imx_4.1.15_2.1.0_ga_alientek
```

如果使用 Ubuntu 自带内核头文件:

```
KERNELDIR := /lib/modules/$(shell uname -r)/build
```

7.2 使用变量使 Makefile 更通用

在 Makefile 开头或通过命令行传入:

```
KERNELDIR ?= /lib/modules/$(shell uname -r)/build
```

命令行覆盖:

```
make KERNELDIR=/path/to/your/kernel
```

?= 赋值的含义:

?= 是条件赋值——只有当变量尚未定义时才赋值。

这样既可以在 Makefile 里给默认值, 又允许用户在命令行用 `make KERNELDIR=...` 覆盖, 非常灵活。